Machine Learning and Pattern Recognition

Transformer-Based Natural Language Processing

Dr. Kewei "Isaac" Li

Department of Digital Engineering
Long Island University

May 1, 2025

Outline

- What is Natural Language Processing (NLP)?
- Introduction
- Tokenization
- Word Embedding
- Transformer

What is Natural Language Processing (NLP)?

- Natural language processing (NLP): a sub-field of AI that uses machine learning to enable computers to understand and communicate with human language
- Transformer models are a very recent family of architectures that have revolutionized fields like NLP, image processing, and generative AI
- NLP is already part of everyday life for many, powering search engines and chatbots for customer service, voice assistants such as Alexa, Siri
- NLP also helps streamline and automate business operations, increases employee productivity

Applications of NLP

Enhanced search: enabling systems to understand the intent behind user queries, providing more accurate and contextually relevant results.

Automation of repetitive tasks:

- NLP-powered chatbots can handle customer queries
- Automatically extract key information and summarize content
- NLP facilitates language translation

Improved data analysis: It enables the extraction of insights from unstructured text data, such as customer reviews, news articles

Powerful content generation: NLP powers advanced language models such as GTP-4 to create human-like text for various purposes.

Outline

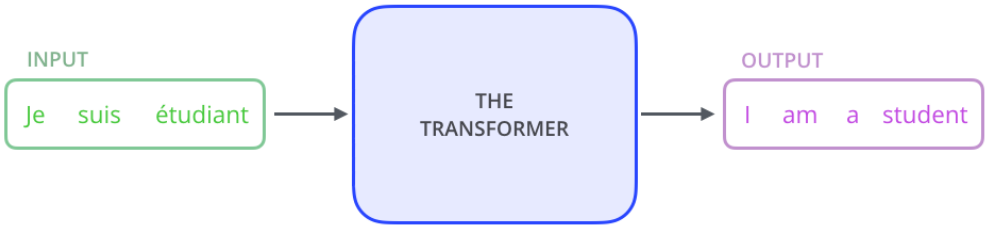
- What is Natural Language Processing (NLP)?
- Introduction
- Tokenization
- Word Embedding
- Transformer

Transformers for NLP

- Human language is inherently sequential in nature (e.g., characters form words, words form sentences, and sentences form articles).
- Prior to transformers, recurrent neural networks (RNNs) briefly dominated the field for their ability to process sequential information
- However, RNNs processed sequential information in an iterative/sequential fashion
- Transformers offer many advantages over RNNs, including their ability to process all items in a sequence in a parallel fashion

Transformers for NLP

- Transformers were introduced in 2017 as a tool for sequence transduction–converting one sequence of data to another
- Example: language translation, sequence completion



Transformers for NLP

- Transformers excel at processing sequences by understanding the relationships between words in context, regardless of their position
- This is achieved by a self-attention layer which allows the model to weigh the importance of each word in a sentence relative to others
- "I poured soda from the can to the cup until it was full." (it = cup)
- "I poured soda from the can to the cup until it was empty." (it= can)

Outline

- What is Natural Language Processing (NLP)?
- Introduction
- Tokenization
- Word Embedding
- Transformer

Tokenization

- To understand “self-attention”, we start with the concepts of tokenization and word embedding
- For NLP models like GPT to understand language, we need to represent written words as numbers – tokenization
- A token is an integer that represents a character, or a segment of characters

Tokenization

Tokenization of "Transformers are very powerful models." by BERT model (Bidirectional Encoder Representations from Transformers):

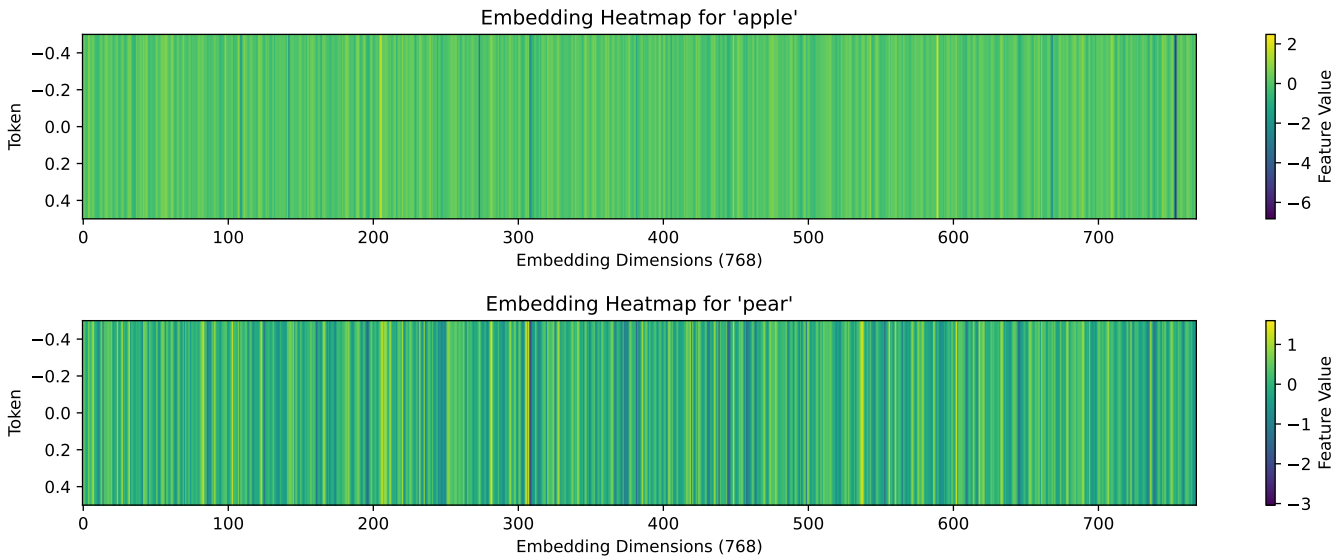
[CLS]	Transformers	are	very	powerful	models	.	[SEP]
101	19081	2024	2200	3928	4275	1012	102

Outline

- What is Natural Language Processing (NLP)?
- Introduction
- Tokenization
- Word Embedding
- Transformer

Word Embedding

- Word embedding: represent words with vectors that capture semantic meaning



Word Embedding

- Word embedding also represents the degree to which any two words are related in the "real-world" to humans
- Words such as "dog" and "cat" should be represented by vectors that are more similar to one another than "dog" and "car" are
- To measure the similarity between two word embeddings, cosine function is often used:

$$\cos(\theta_{\mathbf{u},\mathbf{v}}) = \frac{\mathbf{u} \cdot \mathbf{v}}{|\mathbf{u}||\mathbf{v}|}$$

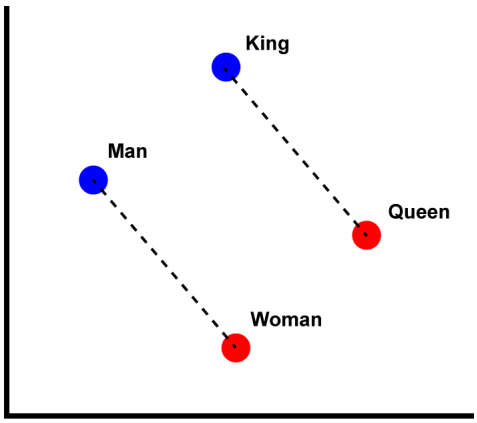
- The cosine similarity is $+1$ when $\mathbf{u} = \mathbf{v}$, and -1 when the two vectors are diametrically opposed to each other

Word Embedding

- Word embeddings have existed for decades, Word2Vec (2012) model is the first approach that achieved great success
- It created a single word embedding for each distinct word in the training corpus
- It enabled all subsequent progress in the field including transformers

Word Embedding

- Word2Vec produced embeddings so effective that many vector relationships reflected real-world semantic similarities



Attention in NLP

- Attention is a mechanism that allows the language model to weigh and combine information from all positions in the input sequence
- It enables models to focus on the most relevant parts for a given task
- Consider a dictionary with keys $\mathbf{k}$ mapped to some values $\mathbf{v}(\mathbf{k})$, now we would like to look up a word ($\mathbf{q}$)
- For example, let $\mathbf{k}$ be the names of some foods, such as apple, pasta, burrito, sushi, etc.
- The corresponding values may be information about the food, such as the type of the cuisine, cost, or ingredients.

Attention in NLP

- Suppose that we want to query by cuisine, e.g., "mexican", instead of looking up foods by name
- Clearly, we will not find the word "mexican" in the dictionary, since it is not a name of the food.
- Instead, we look for "similarity" between vector embeddings of the query and the keys
- The end result represents a probability distribution over the foods, $p(\mathbf{k}|\mathbf{q})$ indicating which $\mathbf{k}$ are best matches for a given query $\mathbf{q}$
- With such a distribution, we can look for keys that are semantically close to the given query.

Attention in NLP

- To compute such distribution, we first embed the word ("mexican") into a query vector, $\mathbf{q} \in \mathbb{R}^{d_k}$, with d_k as the embedding dimension
- Next, suppose our dictionary has n number of entries/keys, we embed each one of these into a key vector $\mathbf{k} \in \mathbb{R}^{d_k}$
- We then obtain the desired probability distribution using a softmax

$$p(\mathbf{k}|\mathbf{q}) = \mathrm{softmax}\left(\mathbf{q} \cdot \mathbf{k}_1, \mathbf{q} \cdot \mathbf{k}_2, \ldots, \mathbf{q} \cdot \mathbf{k}_n\right)$$

- This conditional probability distribution indicates how much attention should be given to each key for a given query

Attention in NLP

- The conditional probability distribution $p(\mathbf{k}|\mathbf{q})$ gives the attention weights, and the weighted average value is the attention output

$$\sum_j p(\mathbf{k}_j|\mathbf{q})\mathbf{v}_j$$

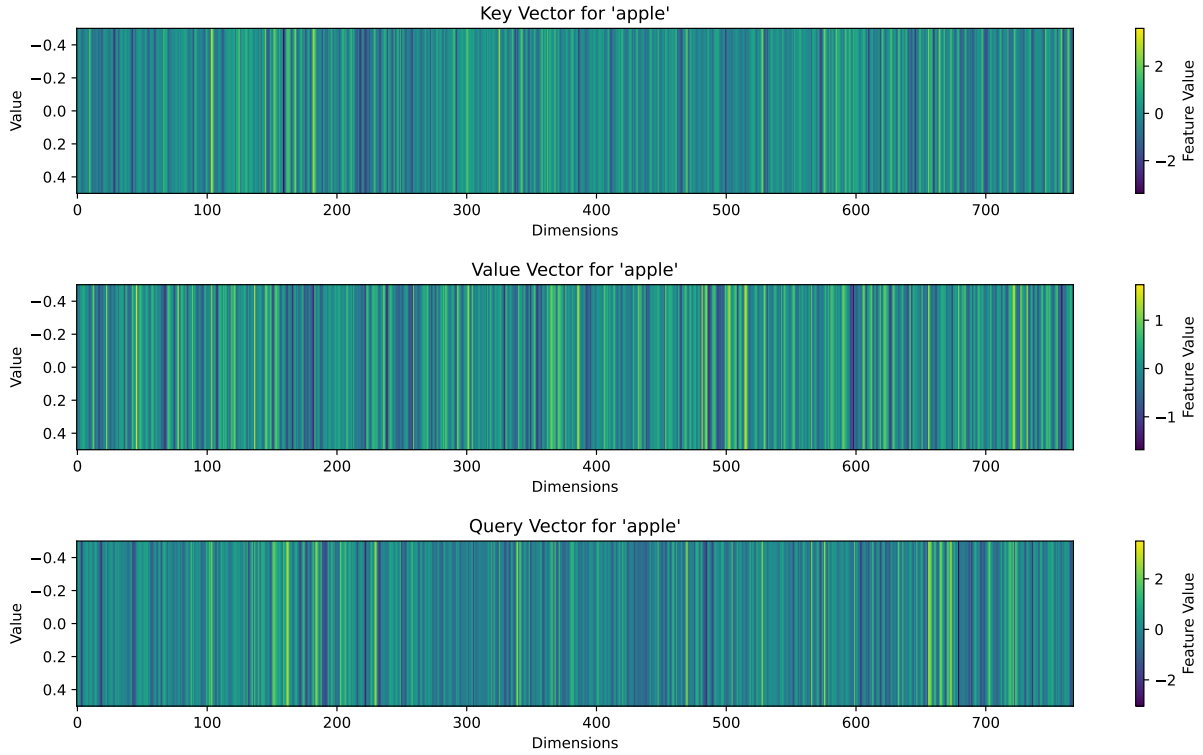
- The meaning of this weighted average value may be ambiguous when the values are just words.
- However, the attention output becomes meaningful when the value are projected in some semantic embedding space

Attention in NLP

- Key vector ($\mathbf{k}$): How the word offers itself to be matched.
- Value vector ($\mathbf{v}$): The actual content the word contributes.
- Query vector ($\mathbf{q}$): How the word seeks relationships with other tokens.

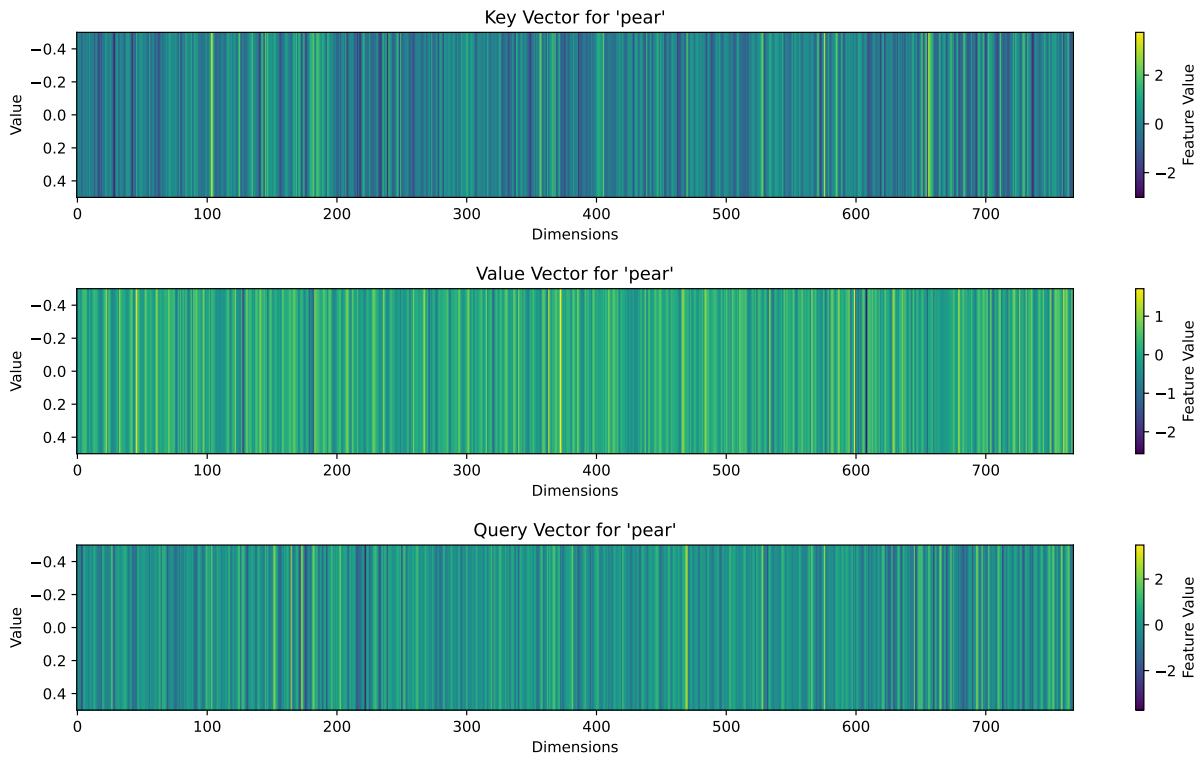
Word Embedding

- Key, value, query vectors of the word "apple"



Word Embedding

- Key, value, query vectors of the word "pear"



Attention

- The weighted-sum idea generalizes to multiple query, key, and values
- Suppose there are n_k number of keys, n_k number of values, and n_q number of queries, we can compute an attention matrix

$$\mathbf{A} = \begin{bmatrix} \mathrm{softmax}\left(\left[\mathbf{q}_1 \cdot \mathbf{k}_1, \mathbf{q}_1 \cdot \mathbf{k}_2, \ldots, \mathbf{q}_1 \cdot \mathbf{k}_{n_k}\right] / \sqrt{d_k}\right) \\ \mathrm{softmax}\left(\left[\mathbf{q}_2 \cdot \mathbf{k}_1, \mathbf{q}_2 \cdot \mathbf{k}_2, \ldots, \mathbf{q}_2 \cdot \mathbf{k}_{n_k}\right] / \sqrt{d_k}\right) \\ \vdots \\ \mathrm{softmax}\left(\left[\mathbf{q}_{n_q} \cdot \mathbf{k}_1, \mathbf{q}_{n_q} \cdot \mathbf{k}_2, \ldots, \mathbf{q}_{n_q} \cdot \mathbf{k}_{n_k}\right] / \sqrt{d_k}\right) \end{bmatrix}$$

Attention

- The entry in i-th row and j-th column of the attention matrix $\mathbf{A}$ is

$$A_{ij} = \frac{\exp\left(\mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{d_k}\right)}{\sum_{j=1}^{n_k} \exp\left(\mathbf{q}_i \cdot \mathbf{k}_j / \sqrt{d_k}\right)}$$

- Then A_{ij} indicates which tokens $\mathbf{x}^{(j)}$ help the most with predicting the corresponding output token $\mathbf{y}^{(i)}$
- The attention output is given by a weighted sum over the values

$$\mathbf{y}^{(i)} = \sum_{j=1}^{n_k} A_{ij} \mathbf{v}_j$$

Self-Attention

- Self-attention is a mechanism where the keys, values, and queries are all generated from the same input
- A typical transformer with self-attention layers maps data from $\mathbb{R}^{n \times d}$ to $\mathbb{R}^{n \times d}$
- In particular, the transformer takes in data (a sequence of tokens) and for each token $\mathbf{x}^{(i)} \in \mathbb{R}^{d \times 1}$, it computes a query $\mathbf{q}^{(i)} \in \mathbb{R}^{d_q \times 1}$, a key $\mathbf{k}^{(i)} \in \mathbb{R}^{d_k \times 1}$ and a value $\mathbf{v}^{(i)} \in \mathbb{R}^{d_v \times 1}$
- In practice, $d_q = d_k = d_v$, and we often use d_k for all three vectors

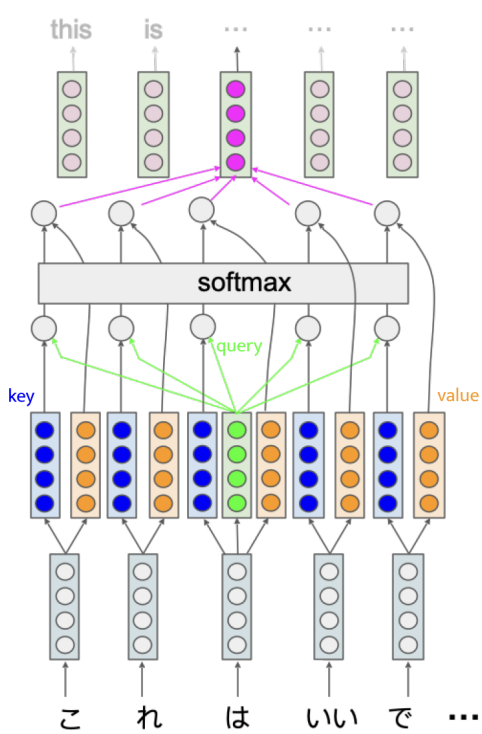
Self-Attention

- The self-attention layer then take in these query, key, and values, and compute a self-attention matrix

$$\mathbf{A} = \begin{bmatrix} \mathrm{softmax}\left(\left[\mathbf{q}_1 \cdot \mathbf{k}_1, \mathbf{q}_1 \cdot \mathbf{k}_2, \ldots, \mathbf{q}_1 \cdot \mathbf{k}_n\right] / \sqrt{d_k}\right) \\ \mathrm{softmax}\left(\left[\mathbf{q}_2 \cdot \mathbf{k}_1, \mathbf{q}_2 \cdot \mathbf{k}_2, \ldots, \mathbf{q}_2 \cdot \mathbf{k}_n\right] / \sqrt{d_k}\right) \\ \vdots \\ \mathrm{softmax}\left(\left[\mathbf{q}_n \cdot \mathbf{k}_1, \mathbf{q}_n \cdot \mathbf{k}_2, \ldots, \mathbf{q}_n \cdot \mathbf{k}_n\right] / \sqrt{d_k}\right) \end{bmatrix}$$

- Since in self-attention, the query, key, and value all come from the same input, we have $n_q = n_k = n_v$, and we often just use n

Self-Attention



- The self-attention output is then given by a weighted sum over the values:

$$\mathbf{y}^{(i)} = \sum_{j=1}^{n} A_{ij} \mathbf{v}_j$$

- Positional information can be added into the vector embedding of each token
- Each self-attention stage is trained to have key, value, and query embeddings that pay attention to some particular feature of the input

Self-Attention

- We generally want to pay attention to many different features of the input data
- For example, in translation one feature might be the verbs, and another might be objects or subjects.
- A transformer utilizes multiple instances of self-attention, known as an "attention head" to capture many different features

Outline

- What is Natural Language Processing (NLP)?
- Introduction
- Tokenization
- Word Embedding
- Transformer

Transformer

- A transformer is the composition of a number of transformer blocks, each has multiple attention heads.
- The goal of a transformer block is to output a rich, useful representation for each input token
- With that, it can be highly efficient for whatever task the model is trained to learn

Transformer

- For simplicity, we assume the transformer internally uses self-attention
- A transformer block is a parameterized function f_θ that maps $\mathbb{R}^{n\times d} \to \mathbb{R}^{n\times d}$ where the input is a sequence of n tokens ($\mathbf{x}^{(i)} \in \mathbb{R}^{d\times 1}$)
- Three projection matrices $\{\mathbf{W}_q, \mathbf{W}_k, \mathbf{W}_v\} \in \mathbb{R}^{d\times d_k}$ are to be learned so that for each token $\mathbf{x}^{(i)}$ we obtain the query, key, and value vectors as

$$\begin{aligned}\mathbf{q}_i &= \mathbf{W}_q^{\mathrm{T}}\mathbf{x}^{(i)}\\ \mathbf{k}_i &= \mathbf{W}_k^{\mathrm{T}}\mathbf{x}^{(i)}\\ \mathbf{v}_i &= \mathbf{W}_v^{\mathrm{T}}\mathbf{x}^{(i)}\end{aligned}$$

- All three vectors have a dimension of $\mathbb{R}^{d_k\times 1}$

Transformer

- If we stack these n query, key, value vectors into matrix form, such that $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$, $\mathbf{K} \in \mathbb{R}^{n \times d_k}$, $\mathbf{V} \in \mathbb{R}^{n \times d_k}$, then we can write,

$$\begin{aligned} \mathbf{Q} &= \mathbf{X}\mathbf{W}_q \\ \mathbf{K} &= \mathbf{X}\mathbf{W}_k \\ \mathbf{V} &= \mathbf{X}\mathbf{W}_v \end{aligned}$$

- These matrices can then be used to compute one (self)attention-layer (attention head) output.

Transformer

- If we use multiple attention heads, then the queries, keys, and values are embedded via encoding matrices:

$$\begin{aligned} \mathbf{Q}^{(h)} &= \mathbf{X}\mathbf{W}_{h,q} \\ \mathbf{K}^{(h)} &= \mathbf{X}\mathbf{W}_{h,k} \\ \mathbf{V}^{(h)} &= \mathbf{X}\mathbf{W}_{h,v} \end{aligned}$$

  where $h = 1, 2, \ldots, H$ is an index of attention heads

- We then perform a weighted sum over all the outputs for each head,

$$\mathbf{u}'^{(i)} = \sum_{h=1}^{H} \mathbf{W}_{h,c}^{\mathrm{T}} \sum_{j=1}^{n} A_{ij}^{(h)} \mathbf{v}_j^{(h)}$$

  where $\mathbf{W}_{h,c} \in \mathbb{R}^{d_k \times d}$, $\mathbf{u}'^{(i)} \in \mathbb{R}^{d \times 1}$, and $i = 1, \ldots, n$ an index of tokens

Transformer

- This is then standardized and combined with $\mathbf{x}^{(i)}$ using a LayerNorm function

$$\mathbf{u}^{(i)} = \mathrm{LayerNorm}\left(\mathbf{x}^{(i)} + \mathbf{u}'^{(i)}; \gamma_1, \boldsymbol{\beta}_1\right)$$

  where $\gamma_1, \boldsymbol{\beta}_1 \in \mathbb{R}^d$ are parameter vectors

- The LayerNorm function transforms a d-dimensional input $\mathbf{z}$ as

$$\mathrm{LayerNorm}\left(\mathbf{z}; \gamma, \boldsymbol{\beta}\right) = \gamma \frac{\mathbf{z} - \mu_z}{\sigma_z} + \boldsymbol{\beta}$$

  where μ_z is the mean and σ_z the standard deviation of $\mathbf{z}$:

$$\mu_z = \frac{1}{d}\sum_{i=1}^{d} z_i, \quad \sigma_z = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(z_i - u_z)^2}$$

Transformer

- To compute the final output, we follow the "intermediate output then layer norm" approach.
- In particular, we first find the transformer block output $\mathbf{z}'^{(i)}$ by

$$\mathbf{z}'^{(i)} = \mathbf{W}_2^{\mathrm{T}} \mathrm{ReLU}\left(\mathbf{W}_1^{\mathrm{T}} \mathbf{u}^{(i)}\right)$$

  where $\mathbf{W}_1 \in \mathbb{R}^{d \times m}, \mathbf{W}_2 \in \mathbb{R}^{m \times d}$ are weights

- This is then standardized and combined with $\mathbf{u}^{(i)}$ to compute $\mathbf{z}^{(i)}$:

$$\mathbf{z}^{(i)} = \mathrm{LayerNorm}\left(\mathbf{u}^{(i)} + \mathbf{z}'^{(i)}; \gamma_2, \boldsymbol{\beta}_2\right)$$

  where parameters $\gamma_2, \boldsymbol{\beta}_2 \in \mathbb{R}^d$.

- These vectors are then assembled to obtain the final output $\mathbf{Z} \in \mathbb{R}^{n \times d}$

Transformer

- The parameters of the transformer block include the weight matrices $\mathbf{W}_{h,q}, \mathbf{W}_{h,k}, \mathbf{W}_{h,v}, \mathbf{W}_{h,c}, \mathbf{W}_1, \mathbf{W}_2$ and the $\mathrm{LayerNorm}$ parameters $\gamma_1, \gamma_2, \boldsymbol{\beta}_1, \boldsymbol{\beta}_2$.
- A transformer is the composition of L transformer blocks, each with its own parameters:
$$\mathbf{Z} = f_{\theta_L} \circ \cdots \circ f_{\theta_2} \circ f_{\theta_1}(\mathbf{X})$$
- The hyperparameters are d, d_k, m, H, and L.

Final Exam Schedule

- THUR 5/8, 6:00 PM - 8:35 PM, Humanities 701

Course Evaluation

- For AI 681 students:
  https://liu.iasystem.org/survey/37885